

Data fragmentation: using AI to rent technology debt or to pay it off

Written with the assistance of a language model and tracked with the Colophon method. The method note, with the contribution percentages, is at the end. This is a translation of the Italian original.

Here are two typical situations.

A B2C services company uses several pieces of software to automate the revenue cycle — lead generation, contact, sale, invoicing, payment, service delivery — with minimal integration between them. As the business grows the systems are evolved quickly, creating application silos, each with its own data, and the correlation between the data in the various silos is no longer unique or guaranteed, making end-to-end process governance difficult (which customer and which lead does that incoming bank transfer refer to?) along with the strategic work of growing the business (how much cash did that campaign actually bring in?). In this case data fragmentation introduces an inefficiency and can hold back the company's ability to scale.

An established B2B services company, handling significant volumes of mission-critical transactions, finds that when operational, privacy or security problems occur it cannot assess in a timely way their impact on people (how many customers, how much internal and partner staff, and in what role), on data (which data, in what form, which processing activities) and on processes (what actions to take, in what timeframe, with what level of authorisation). This is because the number of systems involved, their layering and overlap has created data silos, barriers, functional duplication, often not adequately documented, especially in the case of older data. Extracting a list of affected customers, a map of the internal actors involved, performing a detailed analysis of the data and assessing the impact of a problem becomes a project in its own right, one that can last weeks or months, with uncertain results and costs. In this second case data fragmentation produces a history that can no longer be demonstrated: risk accumulates along with the volume of transactions carried out over time. With the current push on compliance and risk control, having this kind of situation is a genuine time bomb waiting to go off.

These are two faces of the same problem, and it is worth naming it. "Technology debt" — all those situations where a company's technology platform lags behind the state of the art — is one of the daily concerns of a CTO and a CIO, and data fragmentation and data quality are among its most expensive components. What changes, between the two cases, is the currency in which you pay it: efficiency in the first, risk in the second.

The invisible rent: what it already costs today, and why it is becoming visible now

Data fragmentation is a cost, and often an invisible one. Companies in situations similar to those described above certainly pay a significant cost to manage the inefficiencies that result: staff and consultants

dedicated to manual reconciliation, or to writing software "integration" procedures between systems.

It is a line item that appears in no dashboard, because it is not concentrated anywhere: it is a bit of the time of many people, across many different functions. My impression is that in a corporate ICT organisation it is worth a double-digit share of opex. But that is exactly the point: nobody knows precisely, because nobody measures it.

When do these costs surface and become visible? A typical case, in the past, was migration to a new system. Switching on a new CRM or an ERP is emblematic, because it forces you to put the data in order: often the most expensive and longest part of the project is not customising the new software, but the clean-up of the data to be imported from the old one. A vast, immense clean-up.

A "new" case is the use of AI not to solve data fragmentation at source, but to make it tolerable. Instead of reconciling by hand, instead of migrating to systems that cover all processes end-to-end, you build a set of "intelligent" procedures that reconcile the data. On the face of it, it looks like the obvious answer that nobody had thought of: AI can handle situations that are new and not fully defined, it seems purpose-built for interpreting the misalignments between systems.

And it is true, it is an effective solution. But with two problems. First, the problem is not solved but rented: it generates a recurring cost "in tokens" that can become significant — and which, unlike the previous one, is visible. Second, it is a non-deterministic solution, in the sense that it can get things wrong, and in some contexts approximation increases risk instead of reducing it.

Why nobody fixes it: the valley of despair, and the conversation you only win after the damage

I think any CTO can find, for their own company, a path towards an integrated and consistent data architecture, with a performant, resilient, secure and scalable implementation.

But how do you make the case for this kind of project in front of a CEO, a steering committee or a board? The benefits in terms of cost and risk reduction can be significant, but in general you start from an initial situation in which the operating cost is hidden and the company works anyway. The subject is then an exquisitely technical one, and committing the company to a result in the medium term, while still passing through the "valley of despair" typical of any project, is a choice that risks isolating the CTO who proposes it.

The only situation in which all stakeholders raise their level of commitment, paradoxically, is in the face of a problem. If the accumulated risk at some point "explodes" into an incident, the remediation plan can become the way to resolve the technology debt that caused it. Which means that the moment the project becomes approvable is exactly the moment it stopped being an investment and became a repair.

What actually breaks: semantic debt, and the suite as a packaged solution

It is worth being precise about what breaks, because it is not what it seems. The difficulty is not technical: formats, protocols and connectors are things we have been solving for twenty years. The difficulty is semantic. "Customer", in the CRM, is an opportunity that closed; in billing it is a legal entity with a VAT number; in support it is whoever opens the ticket. They are three entities with three different life cycles, which sometimes coincide and sometimes do not. The lead and the customer are born with different keys in different systems, and the link between those keys is often the one thing nobody designed, because at the time it seemed obvious. When that link is lost — and that is exactly what happens in the first of the two cases — you have not lost a piece of data: you have lost the ability to reconstruct a history. Integration projects do not fail on the ETL. They fail in the meeting, when someone has to decide which of the three definitions of "customer" wins.

The problem of data fragmentation is solved structurally by rebuilding the correlations between application silos. If you think about it, that is what happens when a company buys a CRM or an ERP: in the end you are not buying a piece of software, you are buying its data architecture. It is a packaged solution — to have it you also have to take the platform, the migration and the lock-in. And it is a choice that is justified only if there are stronger drivers: operational efficiency, process governance, functional coverage. Solving fragmentation comes as a side effect; it is never the reason you sign.

Today, however, AI unpacks the package: it is possible to rebuild the data architecture without adopting cross-cutting platforms.

What changes now: AI shortens the journey through the valley of despair

Before getting into the substance, a distinction that holds the whole argument together: **AI in the build path, not in the execution path.**

Using a model to reconcile data on every transaction means putting it in the execution path: the cost is a rent that grows with volume, the error happens in production on real data, and the output is the answer itself. Using it to design the architecture, clean up the historical data and write the access procedures means putting it in the build path: the cost is one-off, the error is caught in review, and the output is deterministic — code, schemas, migrations.

It is the same technology. What changes is whether you buy it as an investment or rent it as a service.

Instead of using AI to build "intelligent" integration procedures between systems, it is possible to implement a different strategy.

1) Design a data architecture, that is, a reference semantic model and a common access layer. This is not a database schema: it is the set of rules about which is the authoritative source for each piece of information, how it is accessed, and who decides when it changes.

In the simplest cases it can be a single corporate database, suitably redundant and with all the appropriate security measures. In the more general case it is a set of components (different databases, storage, access services, communication middleware) that guarantee the correctness, uniqueness, distribution and security of the data, and make it accessible where it is needed with performance appropriate to each use case. The important thing is to have a single governance and a clear, robust and fast process for evolving it at the speed of the business. The business applications themselves are part of this architecture: if the authoritative source for a piece of data — the customer master data records, for example — is the corporate CRM, the data architecture will ensure that every copy of that data in other systems is a slave to the company's source of truth.

2) In the short term, all the integration procedures between systems (which certainly exist) are disintermediated by procedures for dialogue between each system and the common data access layer.

3) In the medium term, the individual systems use the data access layer as the reference point for handling any information whose scope of use extends beyond their own application domain.

Now, this approach — "classic", I would venture — has a heavy impact on a company's ICT architecture. But since the difficulty of all these integrations is semantic in nature, this is precisely the field where LLMs can make a decisive contribution. In particular:

The reference semantic model can be very elaborate: some choices depend on in-depth analysis of the data (eliminating redundancies in customer master data records, for example) and the remediation procedures can be very complex. The design and maintenance of the model can be entrusted to a set of specialised AI agents, which maintain and evolve not only the model but above all the context behind it, that is, the real corporate know-how about operational data.

Building the integration procedures with the various systems and providing a data access layer is an activity that requires the development of a considerable amount of code. Here too, the analysis and development can be entrusted to AI agents, with costs and timescales that are not comparable to those of a traditional team.

The agents responsible for managing the architecture can then find, implement and manage every possible optimisation: reducing the number of exposed services, improving performance, running costs and security.

The evolution of the semantic model and of the access layer can be managed in a similar way. Imagine a team that wants to build a new application, which needs corporate data that already exists and adds new data of its own: the architect agent receives the specifications, generates a proposed solution — data plus access services — reusing as much as possible of what is already available, or activating a developer agent to build any new components.

The flip side: the velocity fracture, and vibe coding

Thinking about companies that decide to shift the centre of gravity towards make development of all or part of their application platform, data fragmentation is an enormous risk.

Usually the attention is focused on application functionality, and the data is cloned or carried along. In short: the data strategy does not evolve at the same speed at which new functionality, artifacts and components are produced. It is a genuine fracture in velocity, and there are no culprits: nobody consciously decides to fragment, it is that one side of the system has accelerated tenfold and the other has not.

On data fragmentation there are no public numbers; on code there are, and it is the same fracture seen from the side where somebody is keeping count. GitClear measures software quality by reading the history of git repositories: in its 2026 research it analysed 623 million changes made between 2023 and 2026 — the period in which AI assistants entered development — comparing them with previous years. Two indicators say that a lot is being produced and little is being tidied up: duplication of code blocks has grown by 81%, and refactoring — lines moved rather than added — has fallen from 21% in 2022 to 3.8%. But the two that best describe what I am talking about are others: the share of changes that touch code not updated for more than a year has fallen by 74%, and the number of calls through which new code hooks into existing code has dropped by 35%. Translated: nobody goes back to fix things any more, and the new is born already disconnected. It is the silo, measured.

(This is a vendor's analysis, not a peer-reviewed study, and it does not state its own methodological limits. I take it as a robust indication, not as proof — but it is an indication that anyone developing this way will recognise.)

Vibe coding did not invent the fragmentation problem: it removed the friction that was holding it back.

Building a single data architecture governed by AI agents solves this problem natively. In vibe coding development you activate the virtual architect in charge of the data architecture, which produces what the application needs. And in this case the AI does not clean up the data: it works directly on the access, creation and modification procedures. The choice of how to handle the data does not belong to the application, but to the agentic team that governs the data architecture. What is created, to all intents and purposes, is a negotiation between two teams of agents that have different objectives, partly synergistic and partly conflicting, as is right and proper. On one side the functional and application objective, on the other governance, quality and certainty of the data. This is a dispute between agents, not the typical corporate meeting, in which the winner is often whoever shouts loudest or is friendliest with the CEO. You can define an agentic resolution mechanism, or else the CTO, or a delegate, steps in.

This approach can also be applied to other architectural topics: data fragmentation is the case I am dealing with here, but a similar issue concerns, for example, identity management and authentication and authorisation procedures.

The risk you cannot see: the dashboards are green until the day before

An important point concerns risk management. If you think about the two initial examples, in the end data fragmentation generates costs that in "business as usual" are absorbed and accepted by the company, and often justified by speed of implementation or by management complexity. So, even though there are areas for improvement, from a cost point of view there may well be no alarm at all: all the dashboards look fine.

Risk is quite another matter. It grows without a signal and then turns abruptly into a problem or a catastrophe. I think it is important, when analysing your own data, to capture this kind of information:

- in how many places the same entity is "mastered";
- how many key links between systems cannot be reconstructed automatically;
- what share of the critical flows crosses more than N systems;
- the cost of reconciliation per transaction — which now, in tokens, can genuinely be calculated.

These are four very different quantities, and I do not have a formula to propose: anyone who proposes one to you, at this stage, is selling it to you. What can be done straight away is more modest and more useful. Take the three or four flows that generate the most value and, for each of them, count how many systems it crosses, how many key links cannot be reconstructed automatically, and how much it costs today to reconcile them. These are numbers you can obtain in a few days and that rarely already exist somewhere. The first value of that exercise is not the index: it is that for the first time somebody in the company has a number to put next to a word that until now was only an impression.

Where to start: the to-be architecture as target and as gate

The first thing to do is to define — with the support of AI and using all the information available to the company — the to-be data architecture: the reference semantic model and the access layer.

The implementation is then incremental, and this is the only sensible approach when you develop in vibe coding. It proceeds in two opposite directions: one looks backwards, the other looks forwards.

1) The target: clean up what exists. You identify all the points where data correlation is problematic and address them one by one, using the data architecture as an instrument of disintermediation. The architecture says where you have to get to, and each intervention shortens the distance.

2) The gate: stop the new from adding debt. Instead of building components that duplicate data and create new flows — and that therefore, paradoxically, increase fragmentation — the agents that govern the data architecture are involved in the development of every new object. No new component goes into production without having passed through there: this is what stops the bleeding, and it starts doing so from day one.

In this way you begin to get results in the short term as well. With careful use of an agent architecture the developments become progressive, and the cost can stay "below threshold", without having to ask for an extraordinary investment. This is what makes the approach defensible in front of the CEO: it produces an

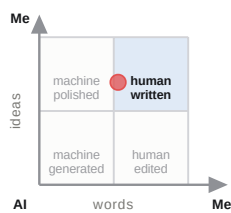
effect before the clean-up is finished. Which, as it happens, no longer shortens the journey through the valley: it avoids it altogether.

Thinking about the two corporate situations mentioned at the start:

The company that grew quickly would build its own data architecture with the same tools it uses to build everything else — in vibe coding — but with objectives other than speed alone: full governance, correctness, timeliness of access to data. Not a separate project: a rule about how you develop.

The established company can start from operating cost reduction alone, without generating organisational trauma and with steady, measurable results at every step. And precisely for that reason it can build consensus from the bottom up, instead of having to ask for it from the top. Which is, once again, a way of crossing the valley without going down into it.

In the shortest possible terms: data fragmentation has never been a problem of technical feasibility, but of project sustainability. Using AI is not about making the debt more tolerable: it is about finally making paying it off something you can propose. Whoever uses it only to tolerate it will pay rent on it, forever.



Method note. I wrote this article with the assistance of a language model, and every intervention was recorded as it happened, using the Colophon method. Content: 69% mine, 31% the AI's. Text: 53% mine, 47% the AI's. The two figures measure different things — the first the ideas, the second the words that express them — and the difference is the interesting part: the AI wrote more words than it brought ideas, because it worked mostly in revision, research and titling. The first draft is 86% mine. The quadrant alongside places the text on the two axes. I answer for every statement in it.

One caveat specific to this version. The measurement above was taken on the Italian original, sentence by sentence, while it was being written. This English text is a translation, produced by the language model and reviewed by me: on the lexical axis it is almost entirely the model's work, and the 53/47 figure does not describe it. The ideas are the same, the words are not. Where the two versions diverge, the Italian prevails — it is the one the register measures.